OLSR and IPv6 using Mininet

This work was prepared by Prof. Manuel Ricardo and Filipe Abrantes and later updated and adapted to Mininet by Eduardo Nuno Almeida and André Coelho.

This laboratory work consists in setting up an IPv6 ad hoc network controlled by the OLSR routing protocol using the Mininet network emulator. This guide is organized into three sections.

The first section explains the Mininet software that will be used to run an emulated network testbed. Read it carefully, as it explains how to use Mininet, as well as how to run OLSR and Wireshark on Mininet.

The final section contains the questions that should be answered in the final report.

1 Mininet

In this work, we will use the Mininet software to emulate a wireless network testbed. Mininet is a network emulator that creates a network of virtual hosts, switches, controllers, and links. Mininet hosts are virtualized in Linux network namespaces, which allows the creation of multiple independent TCP/IP network stacks. Also, since they are running on top of the Linux kernel, all Linux software works exactly in the same way.

More information on Mininet is available on their website: http://mininet.org/overview/

1.1 Mininet Installation

Mininet runs on the Linux kernel. Therefore, in order to set-up the Mininet testbed, we will use a standard Linux distribution (e.g., Ubuntu) on a Virtual Machine (VM) or installed directly on a PC.

The official Mininet website provides a prebuilt VM based on Ubuntu 20.04.1 with all packages already installed. However, this VM does not support IPv6, nor does it contain a graphical user interface.

Therefore, we will install Mininet through Ubuntu's package manager (apt-get). The complete instructions to install Mininet are provided on this website. Ignore the Software Defined Network (SDN) parts (the controller and switch), as they are not used in this work.

http://mininet.org/download/#option-3-installation-from-packages

To install Mininet, run this command on the terminal:

```
$ sudo apt-get update

$ sudo apt-get install mininet xterm
```

To check that everything is working well, run a pingall command on Mininet. This command will create a basic network topology (2 hosts connected through a switch) and perform a ping between both hosts.

```
$ sudo mn --test pingall
```

1.2 Mininet Command Line Interface (CLI)

After starting an instance of Mininet, the Mininet network is controlled through the CLI on the terminal. The Mininet CLI is explained in this website: http://mininet.org/walkthrough/#part-3-mininet-command-line-interface-cli-commands.

In this guide, running commands on the Mininet CLI means that those commands should be run on the Linux terminal where Mininet was launched. This is indicated by the prefix "$ mininet>" on the terminal.

To start a minimal Mininet instance, run these commands (**Note:** Mininet must always be launched with root permissions -- i.e., using sudo):

```
$ sudo mn --help   # See Mininet help and the list of flags

$ sudo mn   # Start Mininet with a minimal topology (2 hosts + 1 switch)
```

After starting a Mininet instance, the network is controlled through the CLI. To see all available commands in the CLI, run this command on the Mininet CLI:

```
$ mininet> help
```

To exit Mininet, run this command on the Mininet CLI:

```
$ mininet> exit
```

There are two methods to run Linux commands on a specific host.

1. Run the command on the Mininet CLI prepending the host on which we want to apply the command. For instance, if we want to run `ifconfig` on host h1:

   ```
   $ mininet> h1 ifconfig
   ```

2. Launch a dedicated terminal for each host. This terminal allows us to run commands directly on that host, without needing to prepend the commands with the host name (e.g., h1, h2, etc.). This is the preferred method for this work, since it allows us to have one terminal per host, similar to a real testbed. To launch a terminal for host 1 and run ifconfig on the newly created terminal, we can simply run this command on the Mininet CLI:

   ```
   $ mininet> xterm h1

   $ ifconfig      # On the newly created terminal
   ```

A useful shortcut to start Mininet with dedicated terminals for every host is to add the flag "-x" when starting a Mininet instance (i.e., `$ sudo mn -x`).

1.3 Association Between Physical and Mininet Network Interfaces

Since Mininet virtualizes each host's TCP/IP network stack, each host has its own network interface. The interface created by Mininet for a host can be seen by running ifconfig on the host's terminal. The convention Mininet uses is "hX-eth0", where X is the host number (e.g., h1-eth0). Therefore, throughout this guide, when referring to the eth0 network interface of hostX, you should instead refer to the Mininet interface "hX-eth0".

Similar to a real testbed, each host in Mininet only has access to its own interface (e.g., h1 can not access h2 interface). However, the Linux PC (outside Mininet) has access to all hosts' interfaces (e.g., s1-eth1, s1-eth2, …).

1.4 Running OLSR in Mininet

In Mininet, only the network stack is virtualized (contrary to a full container). This has two consequences: first, the OLSR daemon (olsrd) only needs to be installed once on the Linux PC, since all hosts share the filesystem; second, the OLSR daemon (olsrd) does not allow the execution of multiple instances on the same machine. The OLSR daemon may check if there is another instance of olsrd running through a temporary lock file created in "`/var/run/olsrd-ipv6.lock`". In order to circumvent this limitation, you must remove this file before starting a new instance of olsrd on another host.

1.5 Running Wireshark in Mininet

Given the association between physical and Mininet network interfaces, there are two methods to run Wireshark on the hosts:

1. Launch Wireshark on the host's terminal (you can continue to use the terminal normally after Wireshark is launched). This method resembles a real testbed, where each host has its own Wireshark.

```
$ sudo wireshark &
```

2. Create a new terminal on Linux (outside Mininet), start Wireshark and capture all the virtual interfaces representing each host (e.g., s1-eth1, s1-eth2, etc.).
   Note: Since we are capturing all interfaces simultaneously, we will need to apply filters when analyzing the logs in order to separate traffic from each host.

```
$ sudo wireshark &
```

1.6 Additional Provided Files

When using Mininet, you should have two additional files provided in Moodle: `mininet_olsr_topology.py` and `mininet_topology_changer.sh`.

The Python script `mininet_olsr_topology.py` defines the network topology to be used in Mininet. The file `mininet_topology_changer.sh` is the updated bash script to change the topology of the network. This script should be run within each host's terminal.

Place these files on a directory of your choice and give the mininet_topology_changer.sh execution permissions.

```
$ sudo chmod +x mininet_topology_changer.sh
```

1.7 Start the Mininet Network

To start a Mininet instance representing the wireless network testbed, run this command on a Linux terminal, where <absolute_path_to_mininet_olsr_topology.py> should be the absolute path to the mininet_olsr_topology.py script file (e.g., /home/eduardo/cmov/mininet_olsr_topology.py).

```
$ sudo mn -x --switch=ovsbr --controller=none
     --custom=<absolute_path_to_mininet_olsr_topology.py>
     --topo=olsr_topo
```

2 Optimized Link State Routing (OLSR)

This section contains the guide for the OLSR lab work.

Hint: You can apply filters in Wireshark when analyzing the captured logs, in order to only show traffic from/to a given IP/MAC, application, and so on.

2.1 Introduction

This work consists in setting up an Ad-Hoc network controlled by the OLSR protocol. For that purpose, we will emulate the Ad-Hoc network using wired interfaces connected to the same switch and blocking connectivity between some of the nodes.

2.2 Network Interfaces Setup

Each group will work with a setup comprising 4 hosts, which represent virtual nodes in a Mininet environment. On these hosts, you are tasked with configuring olsrd. You will have to take the following steps in each one of them. First, make sure you run these commands:

```
$ echo 0 > /proc/sys/net/ipv6/conf/hX-eth0/accept_ra

$ echo 1 > /proc/sys/net/ipv6/conf/hX-eth0/forwarding
```

The first command disables Routing Advertiser (RA) messages. RAs are used as a link local mechanism for neighbor discovery, but because the nodes are going to be routers and use global addresses, RAs are not needed in our scenario. If you want to learn more about it check RFC2462. The second command makes sure that forwarding is enabled. This is a requirement for routing packets.

2.3 OLSR Installation and Configuration

The OLSR daemon (olsrd) can be installed building it manually. To install olsrd and its dependencies manually, run the following commands in a Linux terminal:

```
$ sudo apt install bison

$ sudo apt install flex

$ git clone --depth=1 https://github.com/olsr/olsrd.git

$ cd olsrd/

$ make && sudo make install
```

The OLSR daemon is configured by editing a special configuration file on Linux. A sample of this file is already installed in the directory `/etc/olsrd/olsrd.conf`. To configure OLSR for this work, you should create a configuration file for each host based on the sample. To edit the files, use your favorite editor (e.g., nano, vim, etc.) using sudo.

To create the configuration files, create one copy of the sample configuration file and place it on the same directory, but with the name "olsrdX.conf", where *X* corresponds to the host number. Edit this file with the configurations below. Then, create multiple copies of this edited file for the remaining hosts and change the network interface part.

Note: When using Mininet, remember two things. First, we need separate configuration files for each host (on a real testbed, each PC/host would only have its own configuration file). Second, each host's eth0 network interface must be changed to the Mininet counterpart (e.g., host1 is "h1-eth0", host2 is "h2-eth0", …).

For each host X, please comment/remove any content originally presented in file /etc/olsrd/olsrdX.conf and consider the configuration provided below. The explanation of each command is given in the olsrd's man page and the line comments next to the command. To edit the general OLSR configurations, search the file and add the information indicated below:

```
DebugLevel                  1

IpVersion                   6

LinkQualityLevel            0

MprCoverage                 1
```

To configure the interfaces where OLSR will run, create a new interface block by adding this configuration to the end of the file, replacing the interface name according to the host. Remember that *HelloInterval* is the interval between HELLO messages which are responsible for letting nodes know their neighbors. *TcInterval* is the interval between Topology Control (TC) messages which are responsible for spreading the Ad Hoc network topology to all nodes. *HnaInterval* is the interval between Host and Network Association (HNA) messages responsible for advertising network routes.

```
Interface "hX-eth0"

{

      HelloInterval           6.0

      HelloValidityTime       60.0

      TcInterval              10.0

      TcValidityTime          60.0

      HnaInterval             10.0

      HnaValidityTime         60.0

}
```

2.4 Testing a Static Scenario

Read carefully the information below regarding the network topology.

2.4.1 Configuring the IPv6 Addresses

As mentioned before, each group will have a virtual stand **Y** with 4 hosts running olsrd and connected to a switch. Each host should be configured with a global IPv6 address 202**Y**::**X**/128, where **Y** is the virtual stand id and **X** the host number (e.g. host**2** in stand **3** will have its h2-eth0 configured with the global address 202**3**::**2**/128). This configures a different IPv6 network in each of the nodes, so we will end up with four different IPv6 networks. You can choose any number for the stand **Y** (e.g., 1, 2, etc.).

Setting a global IPv6 address for host **X** in stand **Y**:

```
$ sudo ifconfig hX-eth0 inet6 add 202Y::X/128
```

Additionally, the first host (host1), will have a second interface connected to an IPv6 infrastructured network, which must be configured with a global IPv6 address 3000::**X**/64 (e.g. host1 in stand 3 will have its h1-eth1 configured with the global address 3000::3/64). Another IPv6 router (host5) needs to be set up on that network with the global IPv6 address 3000::254/64. The instructions are in Section 2.6.

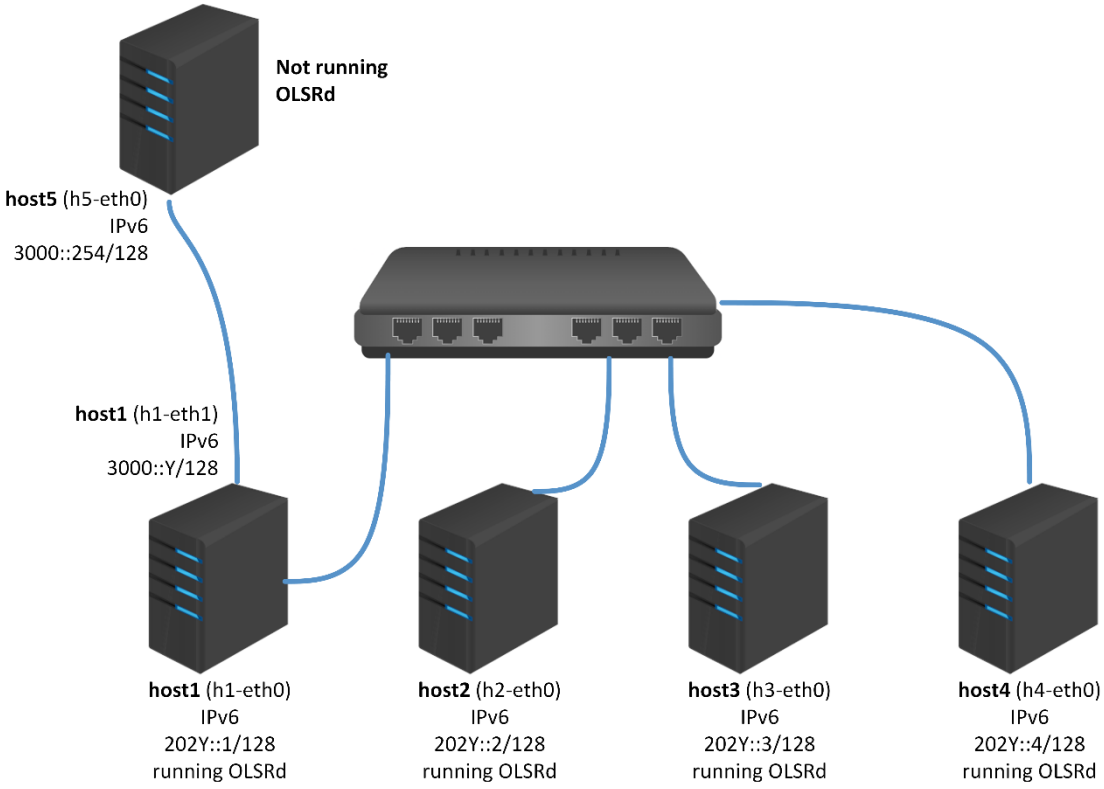
Setting a global IPv6 address for host 1 in Stand **Y**:

```
$ sudo ifconfig h1-eth1 up

$ sudo ifconfig h1-eth1 inet6 add 3000::Y/64
```

2.4.2 Network Topology

The picture below shows the network topology.

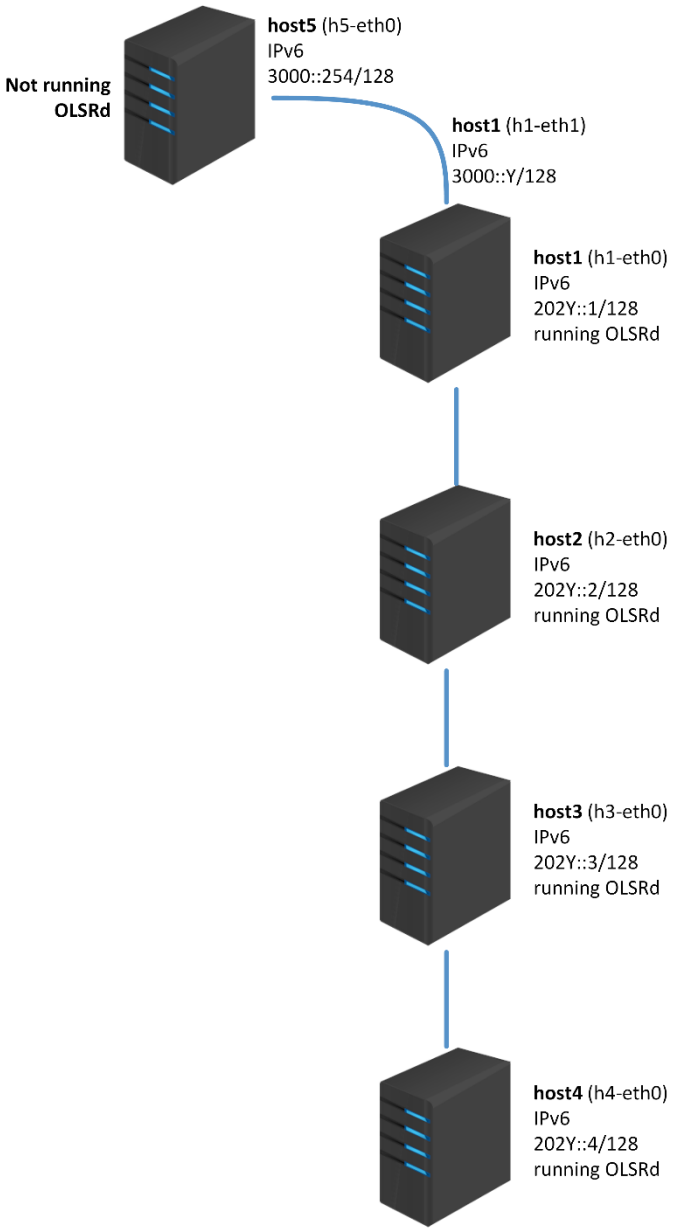


2.4.3 Emulated Ad-Hoc Topology

To emulate the Ad-Hoc topology we want to test, we need to use MAC filtering – we will emulate that a node only "sees" a subset of other nodes by filtering the packets from the nodes we don't want it to "see". This needs special attention when analyzing Ethereal logs as Ethereal will capture those packets. This can also be done using VLANs in network switches. The MAC filtering is done using ip6tables rules.

Note: ip6tables operates at the network layer (IP). Therefore, network packets will still be sent and received between the hosts but are discarded by ip6tables before being processed. Remember this when analyzing the logs in Wireshark.

The picture below represents the first ad hoc scenario we want to emulate, where nodes are all inline:



To automatically set the topology above you can use a bash utility developed for this purpose.

```
Mininet

This script should be run on every host.

Usage: $ ./mininet_topology_changer.sh <topology_number>

     topology_number:

          - 0 for full mesh (every node sees every other node)

          - 1 for line scenario (1-2-3-4)

          - 2 for star topology centered on host 2
```

In case you want to check if everything is ok, execute the following command in the hosts:

```
$ sudo ip6tables -L
```

You should see one MACfilter rule per MAC address to block in the INPUT chain. So, if you execute this in host1, you'll see host3 and host4 MAC addresses blocked. host1 will only "see" host2.

Important: You may want to cross-check the ip6tables rules with the MAC addresses of the hX-eth0 interfaces of your stand. Sometimes the network interface cards are replaced (and so are the MAC addresses) due to hardware failure. If you find any inconsistency, change the topology_changer script and run it again.

2.4.4 The Actual Test

To test the setup, we need to start olsrd on host1, host2, host3 and host4 (the IPv6 router will not run olsrd). To start olsrd, run the following command on each host:

```
$ sudo olsrd -f /etc/olsrd/olsrdX.conf > /dev/null & # Where X is the host number
```

Note: If the configurations are correct, the process will automatically fork and detach from the terminal.

Now check the connectivity from host4 to host1 using the ping6 utility on host4:

```
$ ping6 202Y::1
```

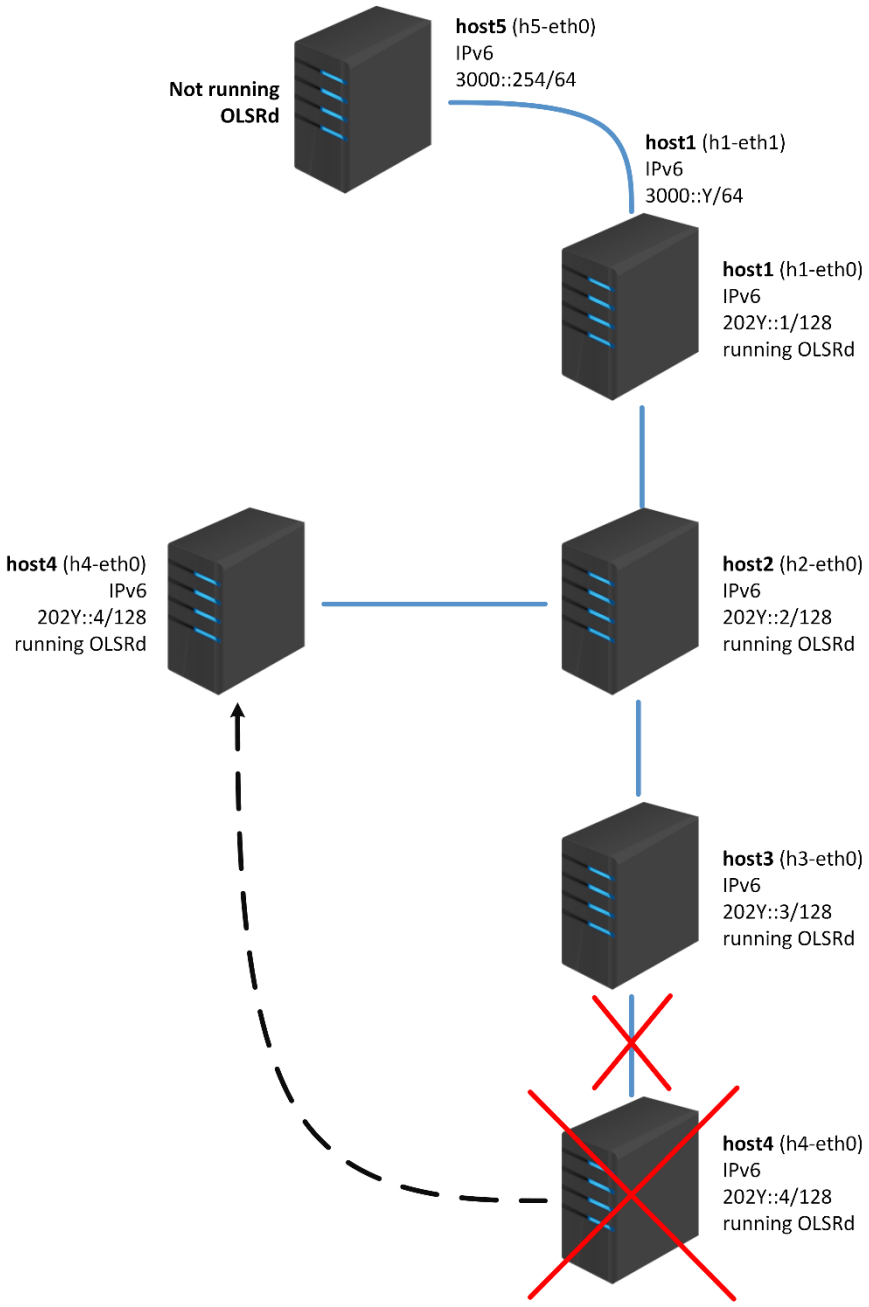
You can list the kernel IPv6 routes by running the route command. Check that the routes to the other hosts are correctly listed (it may take a few seconds to update the routing table):

```
$ route -n6
```

Note: Use Wireshark to capture packets in all the hosts during the test. They will be needed to answer some questions later. You can use ping6 to test/measure the time that two nodes stay connected. You may want to reduce ping6 interval with -i interval.

2.5 Simulating Mobility

Now we are going to simulate mobility scenarios, for example like the one shown below:



To automatically set the topology above you can use the bash utility again. So, if you are on stand 3, to set the above topology you just have to type the following command.

```
On every host:   $ ./mininet_topology_changer.sh 2
```

And it is all set; the routes should be reconfigured in a few seconds (you should measure the time it takes to reconfigure the routes – using **ping6** for instance. You may want to reduce ping6 interval with -i interval) You are welcome to test other mobility scenarios if you want to change the topology_changer.sh script, or/and to alter olsrd parameters (in the configuration file or you can use them as command line parameters) and evaluate their relationship with the time required to reconfigure the routes.

It would also be interesting to measure (not very strict – just to have an idea) the average bandwidth used by the protocol messages in each link (beware of the MAC filtering trick), as well as the size of the messages (especially the HELLO and TC messages sent by host2 before and after host4 has moved).

Note: Use Wireshark to capture packets in all the hosts during the test. They will be needed to answer some questions later.

2.6 Ad Hoc ↔ Infrastructure

Now let's test another feature of the OLSR routing protocol which is the **Home and Network Association (HNA)**. It can be used to establish communication between an Ad Hoc network and an infrastructured one (where nodes aren't running olsrd, just like host5). Let's assume host5 is in the infrastructured network and that host1 is gateway between the Ad Hoc and the infrastructured network.

In host5, we need to set up an IPv6 global address 3000::254/64 and a route for each host of the Ad-hoc network, considering the following commands:

```
$ sudo ifconfig h5-eth0 inet6 add 3000::254/64

$ sudo route -A inet6 add 202Y:0:0::/64 gw 3000::
```

Try pinging host5 from host4:

```
ping6 3000::254
```

This doesn't work (at least if you have followed all the steps of the configuration rightfully), because host4 doesn't have a route for host5. This is when we will introduce the OLSR HNA feature. In host1, open the olsrd configuration file located at "`/etc/olsrd/olsrd1.conf`" and search for the "Hna6" configuration block. If this block exists and the "Internet gateway" configuration is commented out, uncomment it. If the "Hna6" block does not exist or lacks the "Internet gateway" configuration, please add it as presented below:

```
Hna6
{
      # Internet gateway:

      ::    0

      # more entries can be added:

      # fec0:2200:106:: 48

}
```

Restart the olsrd daemon (only at host1). In the Mininet setup, you can list the processes named "olsrd" and stop it by PID (the earliest process should have the lowest PID). Then, restart the olsrd daemon on host1.

```
$ ps -e | grep olsrd

$ sudo kill -9 PID

Note: Replace PID with the actual PID number of the olsrd process associated
with host1

$ sudo olsrd -f /etc/olsrd/olsrdX.conf > /dev/null &

Note: X is the host number (1, in this case)
```

This will cause that all Ad Hoc nodes create a default route using host1 as the gateway. Now try again to ping host5 from host4... it works. You have just integrated an infrastructured network with an Ad Hoc one.

Also try a traceroute from host4 to host5 so you are sure everything is working as expected:

```
$ traceroute6 3000::254
```

If everything is well set up the path will be: host1 → host2 → host3 → host4 (for the line topology).

Note: Use Wireshark to capture packets in all the hosts during the test. They will be needed to answer some questions later.

2.7 Understanding Multi-Point Relayers (MPR)

One of the major features of OLSR for wireless link are the so called Multi-Point Relayers, which are intended to optimize the way relevant information such as Topology Control (TC) or Home and Network Association (HNA) messages are disseminated throughout a MANET. With MPRs, unnecessary transmissions are kept to the minimum possible so that all nodes in the MANET get the information. To know more you can read OLSR RFC and the paper Multipoint relaying for flooding broadcast messages in mobile wireless networks.

In this section, you should test the two scenarios described in sections 2.4 (line topology) and 2.5 (star topology), and for each of them identify which nodes are elected as MPRs (based on Wireshark captures) and argue if these are the ones that you expected or not.

3 Questions

Write a **short report (up to 4 pages, excluding cover)** answering the following questions, including relevant screenshots of Wireshark logs:

1. Imagine that we would use IPv4 and that host1 address was for instance 192.168.10.1/24. What would be the destination IP address of the HELLO packets sent by host1? Assume that the Ipv4Broadcast configuration of the OLSR daemon is not set.

2. Based on the logs collected determine how much bandwidth-per-node does the OLSR protocol consume in both scenarios.
   Hint: you can use Wireshark's tool in "Statistics > Protocol Hierarchy".

   Plot the bandwidth-per-node over time (e.g., bit/s) associated with OLSR traffic in one of the scenarios (either line or star topology).
   Hint: you can use Wireshark's tool in "Statistics > I/O Graphs" to generate a chart for each host.

3. How would you explain the anticipated differences in bandwidth-per-node between the two scenarios, considering how the OLSR protocol operates? Briefly explain how your answer can be verified on the Wireshark logs.
   Hint: take into account the OLSR's Multi-Point Relay (MPR) concept.

4. When node 4 moves, there is a period during which there is no network connectivity. Measure this time period and find a relation between its duration and the *HelloInterval* and *TcInterval* parameters. Identify the specific OLSR messages that were exchanged throughout this process and provide relevant screenshots of your Wireshark captures.

5. Throughout the OLSR guide (section 2), several network configurations had to be made, including routing flags, IPv6 settings and addresses. Taking advantage of the Mininet Python API (see the links in the next section), explain how you can make these network configurations directly on the `mininet_olsr_topology.py` Python script.

6. Imagine you are tasked with emulating a scenario in Mininet where the network topology temporarily changes with respect to traffic flow (e.g., from a line topology to a star topology). However, you are not allowed to add or remove any links nor use the approach implemented by the `mininet_topology_changer.sh` script. Explain conceptually how you would accomplish this task using Mininet's capabilities.
   Hint: remember that Mininet supports Software-Defined Networking (SDN).

4 Links and References

OLSR

Optimized Link State Routing Protocol (OLSR): http://www.ietf.org/rfc/rfc3626.txt

OLSRd Linux daemon: http://www.olsr.org/

Linux IPv6 HOWTO: http://tldp.org/HOWTO/Linux+IPv6-HOWTO/index.html

IPv6 syntax

IPv6 address types

$ man olsrd

$ man olsrd.conf

Mininet

Overview: http://mininet.org/overview/

Walkthrough: http://mininet.org/walkthrough

Python API: http://mininet.org/api/annotated.html